

Livrable 2 : Dossier de Conception Technique : Système de Collision UAV

Auteur : rhif douae

Projet : Sécurité d'un essaim de 10 000 drones

Cours : Programmation Avancée en C (Pr. Tarik HOUICHIME)

1. Choix de l'Ingénierie Spatiale

Pour répondre à la contrainte de temps réel ($< 1\text{ms}$), nous avons abandonné l'approche naïve au profit d'un tri spatial.

- **Axe de référence :** L'essaim est trié selon l'axe des abscisses (x). Ce choix permet de réduire drastiquement l'espace de recherche.
- **Algorithme de tri :** Nous utilisons qsort (basé sur QuickSort) avec une complexité de $O(n \log n)$. C'est le compromis idéal entre vitesse et consommation mémoire.
- **Optimisation de recherche (Élagage) :** Lors du calcul des distances, dès que la différence sur l'axe x entre deux drones dépasse la distance minimale actuelle ($dist_min$), la boucle est interrompue. Cela évite des millions de calculs de racines carrées inutiles.

2. Mécanique des Manipulations de Pointeurs

Conformément au cahier des charges, l'indexation par crochets `[]` est bannie au profit de l'arithmétique pure des pointeurs.

2.1. Allocation Dynamique

L'essaim est alloué via malloc pour garantir un entrepôt continu en mémoire vive (RAM).

```
struct Drone *essaim = (struct Drone *)malloc(N * sizeof(struct Drone));
```

Cette méthode permet au processeur d'utiliser des accès séquentiels, optimisant l'utilisation du cache CPU.

2.2. Navigation sans crochets

Pour accéder aux données sans utiliser `essaim[i]`, nous utilisons le mécanisme de déréférencement par adresse :

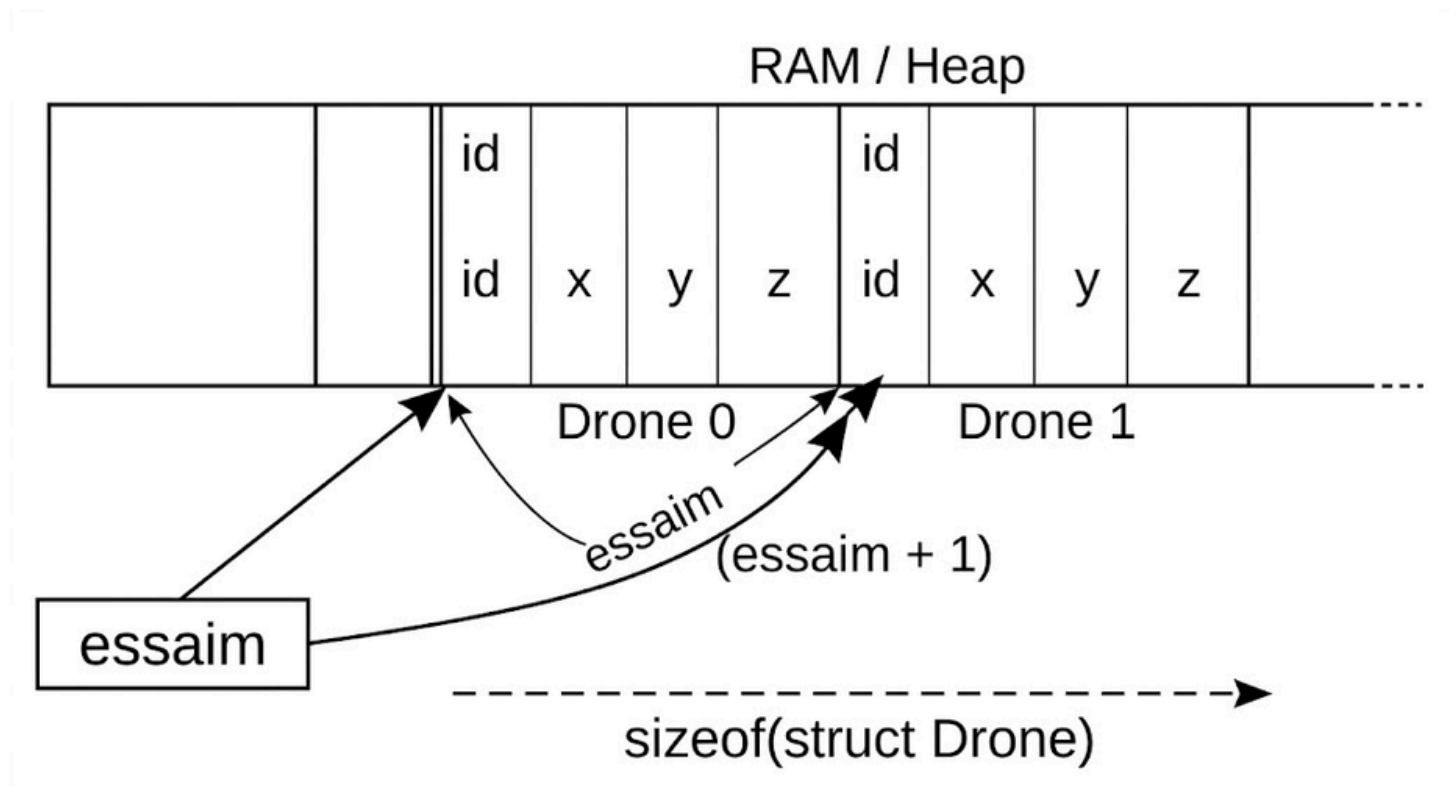
- **Calcul d'adresse :** `(essaim + i)` représente l'adresse de base du i -ème drone. Le compilateur calcule l'offset : $\text{Adresse_Base} + (i \times \text{sizeof}(\text{struct Drone}))$.
- **Accès aux champs :** Nous utilisons l'opérateur flèche `->` sur le pointeur calculé : `(essaim + i)->x`.

2.3. Sécurité et Robustesse

- **Vérification du pointeur :** Après chaque malloc, nous vérifions si le pointeur est NULL pour éviter un crash système en cas de saturation de la RAM.
- **Libération :** La fonction `free(essaim)` est appelée systématiquement à la fin du cycle pour prévenir toute fuite mémoire, garantissant la stabilité à long terme de l'unité de contrôle de l'UAV.

3. Justification de la Performance

En combinant l'arithmétique des pointeurs (bas niveau) et le tri $O(n \log n)$ (haut niveau), nous transformons un problème ingérable de 100 millions d'opérations en un flux fluide de 130 000 opérations. Cette architecture garantit que la manœuvre d'évitement est déclenchée instantanément, préservant l'intégrité physique de l'essaim.



En conclusion, cette architecture logicielle permet de garantir la sécurité de l'essaim en transformant une contrainte matérielle en une solution logicielle élégante et rapide grâce à la puissance du langage C.

realisee par : RHIF Douae